

# Algoritmo de interpolación para aproximación de poblaciones

Simón Díaz Monroy

Noviembre 2025

## 1. Notas de Implementación

### 1.1. Elección de lenguaje de programación

Para el desarrollo del algoritmo se eligió el lenguaje **Python**, debido a su amplia utilización en el ámbito de la computación numérica y científica [1]. Además, ofrece un ecosistema robusto de librerías que facilitan la manipulación de datos, el cálculo matemático y la generación de visualizaciones. En particular, se emplearon las siguientes librerías:

- **Matplotlib**: Librería utilizada para generar gráficas estáticas e interactivas [2]. Se empleó para representar la relación entre los años y la población estimada.
- **Pandas**: Herramienta para el análisis y manipulación de datos [3]. Se utilizó para leer los datos del país asignado desde el archivo de Excel y para generar un nuevo archivo con los resultados de la estimación poblacional.
- **PyInstaller**: Librería destinada a crear ejecutables para distintos sistemas operativos (Windows, macOS y Linux) [4]. Se empleó para empaquetar el algoritmo junto con sus dependencias, de manera que el programa pueda ejecutarse sin requerir la instalación del intérprete de Python ni de las librerías mencionadas.

### 1.2. Organización

La implementación está organizada en tres carpetas principales. La carpeta raíz, denominada **Algoritmo.Interpolación.Lagrange**, contiene el código fuente del proyecto. En su interior se encuentran dos subcarpetas: **in**, donde se almacenan los archivos de entrada utilizados por el programa, y **runnable\_program**, que contiene el ejecutable generado (con sus dependencias) junto con los resultados obtenidos tras la ejecución.

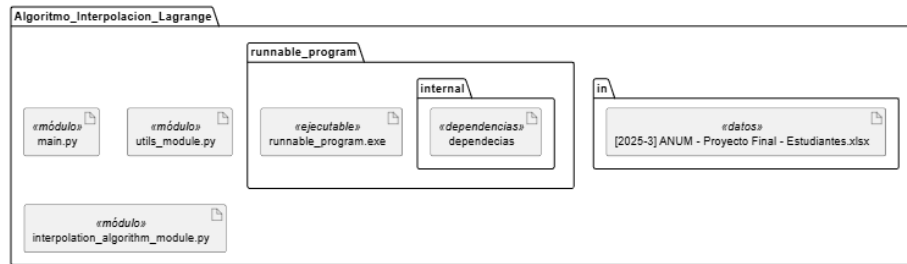


Figura 1: Estructura de la implementación

### 1.3. Captura de los datos

El programa, por defecto, utiliza el único archivo disponible en la carpeta `in` como fuente de datos y selecciona la hoja correspondiente al país 20. El condicional en la línea 4 verifica que el programa se haya ejecutado sin argumentos adicionales. En la línea 7, se realiza la lectura del archivo Excel a partir de la segunda fila (`header=1`), ignorando la primera columna, la cual contiene el nombre del país. La variable booleana `valid` se establece en `True` para indicar que se debe mostrar el menú principal del programa.

```

1      default_name = resource_path("in/[2025-3] ANUM - Proyecto Final -
      ↪ Estudiantes.xlsx")
2      default_sheet = "País 20"
3
4      if len(sys.argv) == 1:
5          name = default_name
6          sheet = default_sheet
7          src_data = pd.read_excel(name, header=1, sheet_name=sheet)
8          valid = True

```

En la función `program_menu(data)`, los valores de cada columna del conjunto de datos se almacenan en dos listas: `years`, que contiene los años, y `populations`, que almacena las poblaciones correspondientes. Después, se recorre la lista de poblaciones para identificar los años que no poseen un valor registrado de población. Los años con datos válidos se agregan a las listas `x_i` y `population_i`, mientras que aquellos sin registro se añaden a la lista `x_opt`, la cual almacena los puntos a interpolar.

```

1      years = data["Año"].tolist()
2      populations = data["Población"].tolist()
3
4      x_i = []
5      population_i = []

```

```

6     x_opt = []
7     for i in range(len(years)):
8         if not math.isnan(populations[i]):
9             x_i.append(years[i])
10            population_i.append(populations[i])
11        else:
12            x_opt.append(years[i])

```

## 1.4. Segmentación del programa

Para implementar el algoritmo se desarrollaron tres módulos principales, cada uno con una responsabilidad específica en el flujo general del programa: manejo de datos de entrada, estimación de poblaciones mediante interpolación y visualización de resultados.

### 1.4.1. main.py

Es el módulo principal del programa. Contiene la función de entrada y controla la ejecución general:

- **resource\_path(relative\_path)**: Devuelve la ruta absoluta de un archivo, compatible con el entorno generado por PyInstaller. Se usa para acceder al archivo de datos empaquetado dentro del ejecutable.
- **proccess\_user\_input()**: Procesa los argumentos de línea de comandos. Permite al usuario especificar el archivo de Excel y la hoja de trabajo a analizar. Si no se proporcionan argumentos, utiliza valores predeterminados. Llama a la función `program_menu()` del módulo `utils_module.py`.

### 1.4.2. utils\_module.py

Encargado de procesar los datos, controlar la interacción con el usuario y generar las gráficas comparativas. Contiene las siguientes funciones y clases:

- **PopulPoint**: Clase de datos (`@dataclass`) que representa un punto de población, con el año, la población y un indicador de si es un dato estimado.
- **output\_aprox(points)**: Genera un archivo de salida en formato Excel con los valores de población estimados y lo guarda en la carpeta `out/`.
- **merge\_interpolation\_points(x\_i, population\_i, x, population\_aprox)**: Combina los puntos originales y los interpolados en una lista ordenada cronológicamente.
- **program\_menu(data)**: Muestra un menú interactivo para seleccionar los años faltantes a estimar. Invoca funciones de `interpolation_algorithm_module.py` para realizar las interpolaciones y llama a `plot_aprox()` para generar las gráficas resultantes.

- `plot_points(ax, known_points, estimated_points)`: Grafica los puntos conocidos e interpolados sobre un eje dado.
- `plot_aprox(points)`: Crea una figura con los datos originales y los estimados. Guarda la imagen como `grafica_ap.png` en la carpeta `out/`.

### 1.4.3. interpolation\_algorithm\_module.py

Contiene la implementación del algoritmo de interpolación de Lagrange y su aplicación sobre listas de valores:

- `lagrange_interpolation(nodes, y_i, x)`: Calcula el valor interpolado de la variable dependiente en el punto `x`.
- `interpolate_by_list(nodes, y_i, x)`: Aplica la función `lagrange_interpolation()` a una lista de valores `x`.

## 2. Componentes del algoritmo de interpolación

El algoritmo de interpolación de Lagrange se basa en la construcción de un polinomio de grado a lo sumo  $n$  que pasa exactamente por los  $n + 1$  puntos conocidos  $(x_i, y_i)$ . Este polinomio se expresa como una combinación lineal de los denominados enésimos polinomios de interpolación de Lagrange  $L_k(x)$ , definidos sobre los nodos  $x_i$  [5].

$$P(x) = \sum_{k=0}^n f(x_k) L_k(x) \quad (1)$$

donde cada polinomio base  $L_k(x)$  se define como [5]:

$$L_k(x) = \prod_{i=0, i \neq k}^n \frac{x - x_i}{x_k - x_i} \quad (2)$$

El algoritmo implementado en Python inicializa primero la variable que contendrá la aproximación final (`approximation`) y una lista para almacenar los polinomios base de Lagrange (1).

---

```

1  def lagrange_interpolation(nodes, y_i, x):
2      """Calcula el valor interpolado en el punto x utilizando el
3      polinomio de Lagrange construido a partir de los nodos y valores
4      conocidos de la función."""
5      approximation = 0
6      l = []

```

---

Posteriormente, el algoritmo calcula los  $n+1$  polinomios base  $L_k(x)$  de grado máximo  $n$ . Para cada nodo  $x_k$ , se evalúan los productos indicados en la ecuación (2), omitiendo el término en el que  $i = k$ . El código correspondiente a esta etapa es:

---

```

1     for k in range(len(nodes)):
2         l_k = 1
3         x_k = nodes[k]
4         for i in range(len(nodes)):
5             if k != i:
6                 l_k *= (x - nodes[i]) / (x_k - nodes[i])
7         l.append(l_k)

```

---

Finalmente, se calcula el valor interpolado de la función  $P(x)$  como la suma ponderada de los valores  $f(x)$  (o  $y_i$ ) multiplicados por sus respectivos enésimos polinomios de interpolación  $L_i(x)$ , según la ecuación (1):

---

```

1     for k in range(len(nodes)):
2         approximation += l[k] * y_i[k]
3     return round(approximation)

```

---

Además, se define una función auxiliar que permite aplicar el método de interpolación a una lista de valores de entrada, devolviendo los resultados en una lista de valores interpolados:

---

```

1     def interpolate_by_list(nodes, y_i, x):
2         return [lagrange_interpolation(nodes, y_i, value) for value in x]

```

---

### 3. Resultados

La interpolación realizada utilizando todos los datos disponibles permitió estimar los valores de población correspondientes a los años sin registro, los cuales se presentan en el Cuadro 1. En el primer año interpolado se observa un valor atípico, producto del *Fenómeno de Runge*. Este fenómeno ocurre cuando se emplean polinomios de interpolación de alto grado, lo que provoca oscilaciones significativas cerca de los extremos del intervalo [6]. En el caso del País 20, el número de nodos utilizados fue  $n = 14$ , lo que explica la magnitud del error observado.

| Año  | Población estimada |
|------|--------------------|
| 1990 | -93774634488       |
| 1998 | 8416669            |
| 2000 | 7902040            |
| 2002 | 7879697            |
| 2012 | 7318728            |

Cuadro 1: Valores de población interpolados para los años sin registro.

Para visualizar de manera más clara el comportamiento de los datos y el valor atípico presente en el extremo del intervalo, se generaron dos representaciones gráficas de la relación Año-Población, mostradas en la Figura 2.

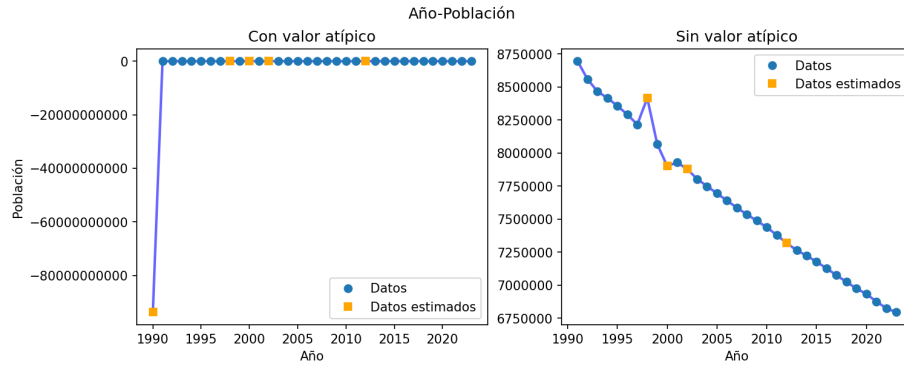


Figura 2: Relación entre el año y la población interpolada.

## Referencias

- [1] P. S. Foundation. (2025) Applications for python. Accessed: 2025-11-03. [Online]. Available: <https://www.python.org/about/apps/>
- [2] M. Developers. (2025) Matplotlib — visualization with python. Accessed: 2025-11-03. [Online]. Available: <https://matplotlib.org/>
- [3] pandas development team. (2025) pandas – python data analysis library. Accessed: 2025-11-03. [Online]. Available: <https://pandas.pydata.org/>
- [4] P. Developers. (2025) Pyinstaller — manual. Accessed: 2025-11-03. [Online]. Available: <https://www.pyinstaller.org/en/stable/>
- [5] R. L. Burden and J. D. Faires, *Análisis numérico*, 10th ed. México: Cengage Learning, 2016.
- [6] C. B. Álvarez, *Capítulo 2. Interpolación y aproximación de funciones*, OpenCourseWare, Departamento de Matemáti-

cas, Estadística y Computación, Universidad de Cantabria, 2025, pp. 26 ff., Documento PDF. [Online]. Available: <https://ocw.unican.es/pluginfile.php/1789/course/section/1349/Capitulo2.pdf>